Audit Logging

who did what, to what, when, from where

Watch the 4-min video

In one line: an audit log is a structured, append-only trail of security-relevant actions (who, what, which resource, when, from where, result), kept apart from app logs, tamper-evident, readable by few, and retained as long as regulations require.

1 Who gave Bob admin?

An account got full admin rights last Tuesday and nobody remembers approving it. Who did it, from where, and was it really them? If the system can't answer, the investigation stalls and the auditor writes a finding. SOC 2, ISO 27001, PCI DSS (Requirement 10) and HIPAA all expect audit trails.

2 What to record

Log **security-relevant actions**: logins and failures, MFA changes, permission and role changes, data exports, configuration changes, deletions, access to sensitive records, and admin actions. Each entry is structured JSON with:

```
{"actor": "alice", "action": "role.granted", "target": "bob", "role": "billing-admin",
"tenant": "acme", "time": "2026-11-03T14:07:22Z", "ip": "203.0.113.7", "request_id": "r-81c",
"result": "ok"}
```

Use UTC timestamps from NTP-synchronised clocks and a correlation ID that links the action to its request trace (lesson 65). Record failures and denials too: a burst of denied requests is often the first sign of an attack. Log identifiers, **never secrets**: no passwords, tokens, keys or full card numbers.

3 Separate from application logs

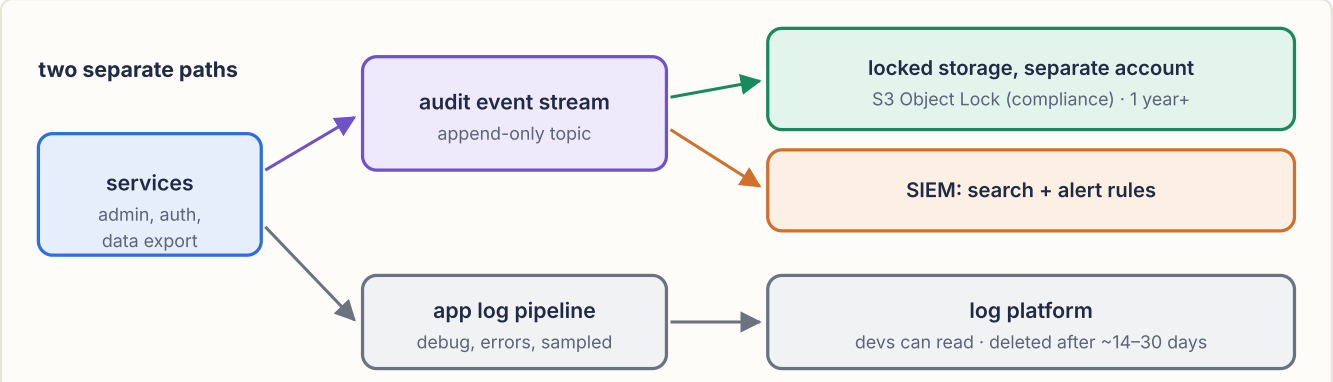


Figure 1. Audit events take their own path to write-once storage and a SIEM; ordinary app logs stay noisy, short-lived and widely readable.

	Application logs	Audit logs
Purpose	debugging, operations	accountability, investigations, compliance
Volume	high, often sampled	low, every event kept
Retention	days to weeks	months to years (PCI DSS: 12 months, 3 immediately available)
Who reads	developers, on-call	a small security / compliance group
Mutability	rotated and deleted freely	append-only, no one can edit or delete

4 Append-only and tamper-evident

If an attacker or rogue admin can delete entries, they cover their tracks. Write to **write-once storage**: S3 Object Lock in compliance mode (not even the root user can delete before retention ends), Azure immutable blob storage, or a separate account the application can only append to. Write asynchronously through a durable queue so auditing doesn't slow requests, but never drop events silently.

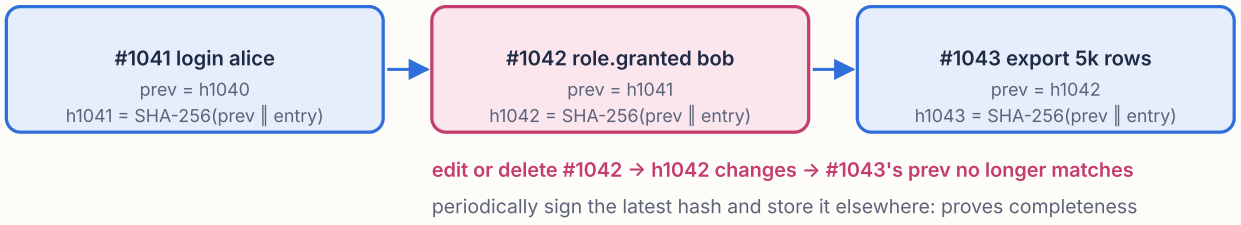


Figure 2. A hash chain makes tampering detectable: each entry includes the hash of the previous one.

Go further with a **hash chain**: each record stores the previous record's hash, so changing or removing any entry breaks verification. Periodically sign the latest hash and keep it elsewhere; that proves the log is complete up to that point. Then **alert** on suspicious patterns: admin grants outside business hours, many exports in an hour, audit-pipeline failures.

WORKED EXAMPLE: RECORDING ALL ADMIN ROLE CHANGES

When Alice grants Bob `billing-admin`, the IAM service writes the event above to an append-only stream before returning success. A consumer copies it to an Object Lock bucket in a separate security account (retention 2 years) and to the SIEM, where a rule alerts on admin grants outside 09:00–18:00 or by accounts that rarely grant roles. Months later, “who gave Bob admin?” is one query, answered in seconds, with a verifiable chain.

COMMON MISTAKES

- Audit events mixed into debug logs and deleted after 14 days.
- Admins who can edit or delete the audit trail.
- Logging tokens, passwords or full request bodies.
- Only logging successes, so failed and denied attempts are invisible.

5 Key terms

Audit log: trail of security-relevant actions for accountability

Append-only / WORM: write once, read many; no edits or deletes

Hash chain: each entry includes the previous entry's hash

Correlation ID: links an audit event to its request trace

SIEM: security information and event management: search + alerts

Retention period: how long records must be kept

6 Interview questions

Q1. What should an audit log contain? BASIC

Who did what, to which resource, when (UTC), from where, with what result, plus a correlation ID; structured, and never secrets like passwords, tokens or card numbers.

Q2. How do you keep it trustworthy? BASIC

Keep it separate from app logs, append-only on write-once storage in a separate account, hash-chained for tamper evidence, and readable only by a small security group.

Q3. Which events would you audit in a banking app? SCENARIO

Logins and failures, MFA and password changes, payee additions, transfers, limit changes, role and permission changes, data exports, and every admin action, including denials.

Q4. The audit pipeline is down. Should user actions fail? SCENARIO

For high-risk actions (admin grants, exports), yes, fail closed or queue durably; for the rest, buffer to durable local storage and alert. Never drop events silently.

Q5. Why isn't storing audit logs in the main database enough? TRICKY

Anyone with write access to that database, including an attacker with the app's credentials, can alter or delete them. They need separate, immutable storage.

Q6. How can you prove no audit entries were deleted? TRICKY

Hash-chain the entries and periodically sign and externally store the latest hash; verification recomputes the chain and detects any gap or edit.

Next: SD-M-018 · Data Privacy and Retention backendroadmap.com